课程讲义 • 2026 年 10 月

从自研网卡到 NVIDIA ConnectX-6

使用差异分析与 CX6 文档地图

为什么“CX6 使用说明”找不到 · 差异通常落在哪 12 个地方 · 一份可直接照做的文档清单

适用对象	网络/云/存储方向的架构师、驱动与系统工程师、售前与运维
课程时长	90 分钟（可扩展为半天工作坊）
版本基线	ConnectX-6 VPI / Dx / Lx；MLNX_OFED 24.10 LTS；DOCA-OFED 3.x；MFT 4.3x
讲义定位	不是命令大全，而是一张“差异地图 + 文档地图 + 上机检查单”

本讲义回答三个问题

1. 为什么按“CX6 使用说明”去搜，总是搜不到完整的配置文档?
2. 自研网卡与 CX6 的差异，通常落在哪些层、哪些文件、哪些工具上?
3. 如果只给 30 分钟上机，按什么顺序检查，才能快速建立全局判断?

讲义结构

按“认知 → 差异 → 文件 → 文档 → 实操 → 课程设计”组织，共 10 章（00–09）。

00	**导读**	三个问题、一页结论、使用建议
01	**为什么“CX6 使用说明”找不到**	家族、分层、平台迁移、版本绑定、KB 文章
02	**两套栈的分层对比**	垂直整合 vs 水平分层，差异的根源
03	**差异通常落在哪里：12 个落点**	认卡、驱动、固件、命名、链路、队列、offload、RoCE、SR-IOV、DPDK、存储、调优
04	**所谓“文件系统参数”**	/sys、/etc、/dev、/proc 的配置落点与持久化
05	**CX6 文档地图**	核心 14 份文档、按任务找文档、版本选择
06	**30 分钟上机检查单**	认卡、看状态、配 IP、验 RoCE、开 SR-IOV、做基线
07	**十二个高频坑**	现象、根因、处理
08	**90 分钟课程设计**	时间表、演示脚本、互动问题、课后作业
09	**附录**	型号对照、命令速查、术语表、参考资料

> **讲师提示**
>
> 第 1—2 章解决“为什么”，第 3—5 章解决“在哪里”，第 6—7 章解决“怎么做”。
>
> 如果只有 45 分钟，讲第 1、3、5 章，并把第 6 章作为课后材料发给大家。

第 0 章 导读：先把问题问对

这门课最容易讲成“CX6 命令大全”，但学员回到机房后仍然会卡住。原因不是命令记得少，而是不知道一件事归哪一层管、该查哪份文档。所以本讲义把“差异”作为主线：先建立分层模型，再把每一类差异落到工具、文件和文档上。

0.1 一页结论

- **CX6 不是一张卡，而是一个家族。**VPI、Dx、Lx 三套硅片、三套固件分支、三套文档；第一步永远是认卡。
- **文档按“层”切，不按“卡”切。**卡手册只讲硬件与安装；RoCE、QoS、SR-IOV 在 OFED 手册；固件参数在 MFT 手册。
- **配置不在一处。**固件参数在 mlxconfig，驱动参数在 modprobe/devlink，接口参数在 ethtool/sysfs，系统参数在 /etc。
- **版本强绑定。**卡型号 ↔ PSID ↔ 固件分支 ↔ OFED/DOCA-OFED ↔ OS/kernel，任何一环错位都会出现“命令对、现象不对”。
- **MLNX_OFED 已进入维护期。**2024-10 是最后一个独立版本，2027-10 EOL；新部署优先 DOCA-OFED，老环境可继续 24.10 LTS。

0.2 怎么用这份讲义

你的角色	建议重点	可以跳过
架构/选型	第 1、2、5 章；型号与版本矩阵	第 6 章的命令细节
驱动/内核工程师	第 3、4、6 章；12 个落点与 sysfs	第 8 章的课程设计
运维/售后	第 3、6、7 章；检查单与高频坑	第 5 章的文档版本细节
讲师	第 8 章；每章的“讲师提示”	附录 C 术语表可作课前阅读

> **一个约定**
>
> 本讲义中的命令默认在 Linux（RHEL/Ubuntu 系）上、以 root 执行；接口名以 ens1f0np0 为例，实际请替换为你机器上的 PF netdev。MST 设备名以 mst status -v 的输出为准。

第 1 章 为什么“CX6 使用说明”找不到

这不是你的检索能力问题，而是 NVIDIA 的文档组织方式与自研网卡完全不同。自研网卡通常“一张卡配一本手册”，而 CX6 的说明被拆成了五类文档：硬件手册、驱动栈手册、固件工具手册、数据面生态文档、支持 KB。如果按“CX6 使用说明”去搜，搜到的往往是产品页、数据手册或过时的博客。

1.1 第一个原因：CX6 是家族，不是型号

“CX6”至少对应三款不同的硅片。它们共享“ConnectX-6”这个名字，但速率、PCIe、加密能力、固件分支和文档入口都不同。拿 Dx 的手册去配 VPI，或者把 VPI 的固件刷到 Dx，都是课堂上真实发生过的事故。

家族	定位	典型速率	PCIe	固件分支	典型 OPN
ConnectX-6 VPI	InfiniBand + Ethernet 双栈，HDR/HDR100	200Gb/s IB；200GbE	3.0/4.0 x16	20.43.x	MCX653106A-HDAT
ConnectX-6 Dx	以太网 SmartNIC，IPsec/TLS/MACsec 卸载	200/100/50/25GbE	4.0 x16	22.4x	MCX623106AN-CDAT
ConnectX-6 Lx	低功耗以太网，Secure Boot	50/25GbE	4.0 x8/x16	26.4x	MCX631102AN-ADAT

注意：上表中的 OPN 只是“典型值”，同一家族有大量 OPN 变体（单/双口、加密/非加密、Secure Boot、液冷、OCP、Socket Direct 等）。最终以 mlxfwmanager --query 输出的 Part Number、PSID 和 Device Type 为准。

上机第一组命令：先认卡

```
# 1) 看 PCI：厂商 ID 15b3，设备 ID 常见 101b / 101d / 101f
lspci -nn | grep -i mellanox

# 2) 看型号、PSID、固件版本（最权威）
mst start
mlxfwmanager --query

# 3) 看内核与固件版本（最快）
ethtool -i ens1f0np0
```

判读要点：Device Type 会直接写 ConnectX6 / ConnectX6Dx / ConnectX6LX；PSID 是 MT_0000000xxx 形式；FW 版本的前缀就是固件分支（20 / 22 / 26）。这三个信息决定了你该看哪本手册、下哪个固件。

1.2 第二个原因：文档按层切，不按卡切

打开《ConnectX-6 Dx Ethernet Adapter Cards User Manual》，你会看到 Interfaces、Hardware Installation、Driver Installation、Updating Adapter Firmware、Troubleshooting、Specifications。它不会告诉你 RoCE 的 GID 怎么配、PFC 怎么开、SR-IOV 的 representor 怎么命名——这些在《MLNX_OFED Documentation》里；而 mlxconfig 的参数含义，又在《NVIDIA Firmware Tools (MFT) Documentation》里。

你想解决的问题	该看的文档	不该只看
板卡尺寸、接口、LED、供电、上架	ConnectX-6 / Dx 用户手册	OFED 手册
装驱动、选 OFED 版本、Secure Boot	MLNX_OFED / DOCA-OFED 安装章节	卡手册的 Driver Installation（只有索引）
RoCE、PFC、ECN、QoS、GID	OFED 手册 Ethernet Network 章节	卡手册
SR-IOV、switchdev、representor	OFED 手册 Virtualization 章节	卡手册
mlxconfig 参数、固件升级	MFT 手册 + 固件发布说明	OFED 手册
DPDK、OVS、NVMe-oF 卸载	DPDK mlx5 文档 / OVS 文档 / SPDK 文档	卡手册
“为什么现象不对”	OFED Troubleshooting + 支持 KB	只看命令示例

1.3 第三个原因：平台与命名正在迁移

过去十年，Mellanox 的文档入口经历了三次变化：docs.mellanox.com → docs.nvidia.com/networking → networking-docs.nvidia.com。很多中文博客里的链接已经 404，但命令仍然“看起来能用”，于是给人造成“文档找不全、命令又对不上”的错觉。

- **MLNX_OFED → DOCA-OFED。**2024 年 10 月是 MLNX_OFED 最后一个独立版本；之后进入 3 年 LTS（到 2027-10），只修关键 bug 与安全漏洞，不再加新功能。DOCA-OFED 是它的 1:1 替代，命令、工具、目录结构基本一致。
- **MLNX_EN 退役。**以太网精简版 MLNX_EN 也不再单独发布；需要更小的包时，用 inbox 驱动或 DOCA-RoCE profile。
- **文档版本选择器。**同一份手册有 5.9、24.10、3.x 等多个版本。看错版本，参数名和默认值都可能不同。

两个必须记住的入口：

文档总入口：`https://docs.nvidia.com/networking/` （产品 → 软件 → 驱动/固件/工具）

新文档平台：https://networking-docs.nvidia.com/ （可直接访问 .md 页面，适合脚本抓取）

1.4 第四个原因：版本是强绑定的

自研网卡通常“驱动包 + 固件”一起发，版本号内部对齐；CX6 则有四条独立的版本线，必须自己对齐：

```
卡型号 / PSID
   └── 固件分支（CX6 20.x / CX6 Dx 22.x / CX6 Lx 26.x）
         └── 固件版本（如 20.43.8016 / 22.47.3576）
               └── 驱动栈（inbox mlx5_core 或 DOCA-OFED / MLNX_OFED 24.10）
                     └── OS / kernel（RHEL 8/9/10、Ubuntu 20/22/24、SLES 15…）
                           └── 功能（RoCE、SR-IOV、IPsec、DPDK…）
```

所以“CX6 能不能用某个功能”不是一个是非题，而是一个五元组问题：哪张卡、哪个固件、哪个驱动、哪个内核、哪个应用。查兼容性时，先看固件发布说明的 “Driver Software, Tools and Switch Firmware” 表，再看 OFED/DOCA 的 Release Notes。

1.5 第五个原因：大量“怎么做”在 KB，不在手册

NVIDIA 把很多“配置示例、排障步骤、性能调优”放进了 Enterprise Support 的知识库文章（KB）。这些文章不在手册的目录里，搜索引擎也不一定收录；但它们是官方内容，质量通常高于博客。建议把下面几篇加入浏览器书签：

- Performance Tuning for NVIDIA Adapters 打开
- Recommended Network Configuration Examples for RoCE Deployment 打开
- Understanding mlx5 Linux Counters and Status Parameters 打开
- Understanding mlx5 ethtool Counters 打开
- HowTo Configure Trust State on NVIDIA Adapters 打开
- HowTo Configure SR-IOV for ConnectX-4/5/6 with KVM (Ethernet) 打开

1.6 第六个原因：检索词不对

中文社区里“CX6 使用说明” “CX6 配置手册”这类词，命中的往往是产品页或旧博客。NVIDIA 的文档标题非常具体，用英文的“产品名 + 文档类型”去搜，命中率会高一个数量级。

你想找	不建议搜	建议搜（精确短语）
卡的手册	CX6 使用说明	ConnectX-6 Dx Ethernet Adapter Cards User Manual
驱动安装	CX6 驱动	NVIDIA MLNX_OFED Documentation 24.10 installation
固件参数	CX6 配置	NVIDIA Firmware Tools mlxconfig
链路/光模块	CX6 光模块	mlxlink utility / mlxcables

你想找	不建议搜	建议搜（精确短语）
RoCE	CX6 RoCE 配置	MLNX_OFED RDMA over Converged Ethernet RoCE
性能调优	CX6 调优	Performance Tuning for NVIDIA Adapters
计数器/丢包	CX6 丢包	Understanding mlx5 Linux counters

本章小结

找不到文档，通常不是“没有”，而是“不在你以为的那一层”。记住三句话：

① 先认卡（VPI/Dx/Lx）；② 按层找文档（卡/驱动/固件/生态/KB）；③ 用英文精确标题搜。

第 2 章　两套栈的分层对比：差异的根源

把两套栈画在一起，差异就很直观：自研网卡是“垂直整合”，一颗芯片、一套驱动、一个工具、一份配置；CX6 是“水平分层”，硬件、固件、驱动、配置工具、数据面生态各自独立演进。这不是谁好谁坏，而是两种工程模式的必然结果。



图 1　自研网卡的“单入口”与 ConnectX-6 的“分层多入口”

2.1 六层对照

层	自研网卡典型	ConnectX-6 标准	差异根源
L5 运维面	厂商 CLI / 私有 REST / 单一日志	ethtool、devlink、sysfs、mlxlink、mlnx_perf（旧）	标准内核接口 + 厂商诊断工具并存
L4 配置面	私有配置文件 + 厂商工具	mlxconfig、mlnx_qos、iproute2、tc、udev	固件、驱动、OS 三层各有各的配置
L3 数据面	自研 PMD / 内核驱动 / 私有流表	RDMA verbs、DPDK mlx5、OVS ASAP²、NVMe-oF	一套硬件，多种数据面入口
L2 驱动	自研驱动，与固件强绑定	mlx5_core（inbox 或 DOCA-OFED）	OS inbox 与厂商栈两条线
L1 固件	私有格式，随驱动包发布	MFT 工具链（mlxfwmanager/flint/mlxconfig）	固件是独立可配置层
L0 硬件	自研 ASIC / FPGA	CX6 VPI / Dx / Lx	一个品牌，三个硅片

2.2 三个直接后果

后果一：排障入口变多

自研网卡出问题，通常看一个厂商工具的输出就够了。CX6 出问题，至少要同时看四个地方：ethtool -S（netdev 统计）、/sys/class/infiniband/.../counters（RoCE 统计）、mlxlink（物理层）、devlink health / dmesg（驱动与固件健康）。只看一个，很容易把“物理层误码”误判成“驱动丢包”。

后果二：配置不再有单一文件

自研网卡的配置往往集中在一个 /etc/xxx.conf；CX6 的配置分散在：固件（mlxconfig，掉电保留）、驱动（modprobe.d、devlink param）、接口（ethtool、ip、sysfs）、系统（sysctl、limits、udev、NetworkManager）。更麻烦的是，很多配置重启后会丢，需要自己写 udev 规则或 systemd 服务固化。第 4 章会逐条列出。

后果三：版本必须自己对齐

自研网卡的驱动、固件、工具通常是一个包，版本天然一致。CX6 的 inbox 驱动随内核走，DOCA-OFED 按季度发版，固件按 LTS 分支发版；三者交叉组合。于是“同一个命令，在 A 机器上行、在 B 机器上不行”是常态，而不是异常。

> **讲师提示**
>
> 这一章不要急着讲命令。先让学员接受一个观点：CX6 的“复杂”主要不是硬件复杂，而是分层带来的“入

> 口多、版本多、文件多”。后面的 12 个落点，都是这句话的展开。

第 3 章 差异通常落在哪里：12 个落点

这一章是课程的主体。我们把“自研网卡 → CX6”的差异拆成 12 个可操作的落点。每个落点都回答四个问题：典型现象是什么、自研怎么做、CX6 怎么做、对应哪份文档。讲课时刻按“现象 → 命令 → 文档”三步走，不要逐条念命令。



图 2 12 个差异落点：前 4 个解决“你是谁”，中间 4 个解决“怎么配”，后 4 个解决“怎么用”

3.1 认卡与型号：先确定你手里是哪一张

典型现象：只知道“这是 CX6”，但实际可能是 Dx 或 Lx；下载了 VPI 的固件，刷不进去；或者照着 Dx 的手册找 IPsec 配置，结果卡根本不支持。

自研网卡的典型做法	ConnectX-6 的标准做法
看厂商型号、一个型号一本手册；固件与驱动同一个包。	三层识别：PCI 设备 ID → Device Type/PSID → 固件分支；三者共同决定文档与固件。

```
lspci -nn | grep -i mellanox
# 15b3:101b 常见于 CX6；15b3:101d 常见于 CX6 Dx；15b3:101f 常见于 CX6 Lx

mst start
mlxfwmanager --query
# 重点看：Device Type / Part Number / PSID / FW

ethtool -i ens1f0np0
# 重点看：driver、version、firmware-version、bus-info
```

判读：Device Type 会直接写 ConnectX6 / ConnectX6Dx / ConnectX6LX；PSID 形如 MT_0000000224；FW 的第一段就是分支（20/22/26）。把这三个值记到检查单上，后面所有文档和固件都以它为准。

对应文档：ConnectX-6 VPI 用户手册 https://networking-docs.nvidia.com/connectx6vpihw/

对应文档：ConnectX-6 Dx 用户手册 https://networking-docs.nvidia.com/connectx6dxhw/

3.2 驱动安装：inbox 还是 DOCA-OFED

典型现象：ethtool -i 的 version 字段一会是内核版本、一会是 5.9/24.10；机器上既有 inbox 的 mlx5_core，又装了 OFED 的包；升级内核后网卡起不来。

自研网卡的典型做法	ConnectX-6 的标准做法
厂商提供一个驱动包（rpm/deb），与固件版本绑定；装完即用。	两条路：OS inbox 的 mlx5_core，或 NVIDIA 的 DOCA-OFED/MLNX_OFED。二选一，不要混装。

```
ethtool -i ens1f0np0
ofed_info -s                  # 有输出说明装了 OFED/DOCA-OFED
modinfo mlx5_core | head -20
dmesg | grep -i mlx5 | tail -30

# 判断当前内核模块来自哪里
modinfo -n mlx5_core
ls -l /lib/modules/$(uname -r)/updates/dkms/ 2>/dev/null | grep mlx5
```

判读：ofed_info -s 有输出（如 MLNX_OFED_LINUX-24.10-1.1.4.0）说明走的是 NVIDIA 栈；没有输出、且 modinfo 指向 /lib/modules/.../kernel/...，说明是 inbox。混装的典型症状是“ethtool -i 显示的版本和 ofed_info 对不上”。

对应文档：MLNX_OFED 24.10 安装 https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/installation.md

对应文档：MLNX_OFED → DOCA-OFED 迁移指南 https://networking-docs.nvidia.com/doca/archive/3-2-3/mlnx_ofed-to-doca-ofed-transition-guide

3.3 固件升级：PSID 必须匹配

典型现象：只升级了驱动，固件还是旧版；或者下载了 CX6 的固件，试图刷到 Dx 上；刷完没有 mlxfwreset，ethtool -i 仍然显示旧版本。

自研网卡的典型做法	ConnectX-6 的标准做法
厂商提供一个刷机工具，固件文件与型号一一对应；刷完重启。	MFT 工具链：mst → mlxfwmanager 查询/升级 → mlxfwreset 生效；也可用 mlxup、flint。

```
mst start
mst status -v
mlxfwmanager --query

# 升级（文件名与版本仅作示例，必须用与你 PSID 匹配的 .mfa）
mlxfwmanager -u -d /dev/mst/mt4123_pciconf0 \
  -i fw-ConnectX6-rel-20.43.8016.mfa

# 不重启使新固件生效（先 query 看支持的 level/type）
mlxfwreset -d /dev/mst/mt4123_pciconf0 query
mlxfwreset -d /dev/mst/mt4123_pciconf0 reset -y

# 验证
ethtool -i ens1f0np0
```

注意：① 固件文件名里的 20.43 只是分支示例，不要照抄；② 升级前读固件发布说明的兼容表；③ 生产环境先备份 mlxconfig（见 3.4）；④ 跨大版本升级可能需要冷重启，mlxfwreset 会提示。

对应文档：MFT / mlxfwmanagerhttps://networking-docs.nvidia.com/mftswum/434118lts/mlxfwmanager-firmware-update-and-query-tool

对应文档：mlxfwresethttps://networking-docs.nvidia.com/mftswum/434118lts/mlxfwreset-loading-firmware-on-5th-generation-devices-tool

3.4 固件参数：mlxconfig 不是 ethtool 的替代品

典型现象：想开 SR-IOV、改端口类型、调 CQE 压缩，却在 /etc 下找不到配置文件；或者用 ethtool 改了，重启就丢。因为这些是固件级参数，归 mlxconfig 管。

自研网卡的典型做法	ConnectX-6 的标准做法
私有配置文件 + 厂商工具；改完重启生效；配置文件可备份。	mlxconfig 直接读写固件 NV 配置；q 查询、set 修改、backup 备份、show_confs 看全部参数。

```
DEV=/dev/mst/mt4123_pciconf0

mlxconfig -d $DEV q
mlxconfig -d $DEV show_confs

# 常见参数（不同固件支持的集合不同，以 show_confs 为准）
mlxconfig -d $DEV set SRIOV_EN=1 NUM_OF_VFS=8
mlxconfig -d $DEV set LINK_TYPE_P1=ETH LINK_TYPE_P2=ETH
mlxconfig -d $DEV set CQE_COMPRESSION=1

# 改前备份，改后确认
mlxconfig -d $DEV -f /root/cx6-backup.conf backup
mlxconfig -d $DEV q | head -40
```

注意：① 改完需要重启或 mlxfwreset 才生效；② 不是所有参数都能改，show_confs 会列出当前固件支持的项；③ 文本值（ETH/IB、True/False）和数字值都支持；④ 这是固件配置，不要和 ethtool 的运行时配置混为一谈。

对应文档：MFT / mlxconfighttps://networking-docs.nvidia.com/mftswum/434118lts/using-mlxconfig

3.5 设备命名：一张卡至少五个名字

典型现象：ip link 看到 ens1f0np0，ibv_devinfo 看到 mlx5_0，虚拟机里看到 ens1f0v0，OVS 里又看到 pf0vf0；不知道谁是谁，日志对不上。



图 3 同一个物理口在 PCI、netdev、RDMA、VF、representor、MFT 六个视角下的名字

```
# 从 PCI BDF 反查 netdev
ls -l /sys/class/net/ | grep 0000:3b:00

# netdev ↔ RDMA 设备
ibdev2netdev
rdma link show

# MFT 设备名
mst status -v

# switchdev 下的 representor
devlink port show
ip -d link show | grep -E "portname|switchid"
```

判读：命名规则通常是 <PCI 槽位>f<功能号>np<端口号>，例如 ens1f0np0；np0 表示网卡端口 0，不是“第 0 个 PF”。VF 的 netdev 常见为 ens1f0v0；representor 常见为 pf0vf0。不同发行版的命名策略不同，最终以 /sys/class/net 的 symlink 为准。

对应文档：OFED / Ethernet Interface https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/ethernet-interface.md

3.6 链路与光模块：物理层问题先看这里

典型现象：链路起不来、频繁 down/up、误码率高、光模块不识别；驱动日志里只有“link down”，没有原因。

```
ethtool ens1f0np0
ethtool -m ens1f0np0                # 光模块 EEPROM
ethtool --show-fec ens1f0np0        # FEC 模式

mlxlink -d /dev/mst/mt4123_pciconf0 -m -c
mlxlink -d /dev/mst/mt4123_pciconf0 --show_fec
mlxlink -d /dev/mst/mt4123_pciconf0 --show_eye

mlxcables -d /dev/mst/mt4123_pciconf0_cable_0 -q
mlxcables -d /dev/mst/mt4123_pciconf0_cable_0 --DDM
```

注意：① mlxlink 是诊断工具，不适合持续轮询；② 网卡用 MST 设备区分端口，不要用 -p（-p 是交换机的 label port）；③ 端口 2 常见为 /dev/mst/mt4123_pciconf0.1；④ 光模块信息优先看 mlxcables 的 DDM，比 ethtool -m 更完整。

对应文档：mlxlinkhttps://networking-docs.nvidia.com/mftswum/434118lts/mlxlink-utility

对应文档：mlxcableshttps://networking-docs.nvidia.com/mftswum/434118lts/mlxcables-cables-tool

3.7 队列、中断、ring、coalesce

典型现象：单流性能上不去、多核 CPU 只有几个核在跑、延迟抖动；不知道该调队列数、ring 大小还是中断合并。

```
ethtool -l ens1f0np0                # 当前/最大列数
ethtool -L ens1f0np0 combined 8     # 改队列（会重置 RSS 表）
ethtool -g ens1f0np0                # ring 大小
ethtool -G ens1f0np0 rx 4096 tx 4096
ethtool -c ens1f0np0                # coalesce
ethtool -C ens1f0np0 adaptive-rx off rx-usecs 32

cat /proc/interrupts | grep -E "ens1f0np0|mlx5_comp"
```

判读：mlx5 在加载时就分配好所有队列的 IRQ，名字形如 mlx5_comp<x>@pci:0000:3b:00.0；实际启用的队列会重命名为 ens1f0np0-<x>。改队列数不会重新分配 IRQ，但会重置 RSS 表，所以调完队列要重新检查 ethtool -x 的 indirection table。

对应文档：OFED / Ethtoolhttps://networking-docs.nvidia.com/mlnxofedswum/24101140lts/ethtool.md

3.8 Offload：软件能看到的不等于硬件在做

典型现象：ethtool -k 显示 tso/gso 都是 on，但 CPU 还是很忙；OVS 显示有流表，但吞吐没有提升；tc 规则加上了，包却没走硬件。

```
ethtool -k ens1f0np0
ethtool --show-priv-flags ens1f0np0
ethtool -K ens1f0np0 gro on gso on tso on

# 用 tc 加一条“只走硬件”的规则
tc qdisc add dev ens1f0np0 ingress
tc filter add dev ens1f0np0 ingress protocol ip prio 1 flower \
  skip_sw dst_ip 192.0.2.10 action drop
tc -s filter show dev ens1f0np0 ingress
```

判读：skip_sw 表示“只下发硬件，不装软件备份”；如果硬件不支持，规则会直接失败，而不是悄悄退回软件。OVS 场景用 ovs-appctl dpctl/dump-flows type=offloaded 确认是否真的 offload；只看到 dpctl/dump-flows 有流，不代表走了硬件。

对应文档：OFED / Ethtool https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/ethtool.md

对应文档：OFED / OVS ASAP² https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/ovs-offload-using-asap2-direct.md

3.9 RoCE 与 QoS：PFC 必须端到端

典型现象：RDMA 能连上，但一跑高吞吐就丢包、重传；主机上配了 PFC，交换机没配；或者配了 DSCP，网卡还在按 PCP 信任。

```
ibv_devinfo -v
ibdev2netdev

# 设置 RDMA-CM 的 RoCE 模式（1=v1, 2=v2）
cma_roce_mode -d mlx5_0 -p 1 -m 2

# 看 GID 表（RoCEv2 的 GID 类型为 v2）
cat /sys/class/infiniband/mlx5_0/ports/1/gid_attrs/types/*
cat /sys/class/infiniband/mlx5_0/ports/1/gids/*

# QoS / PFC / trust
mlnx_qos -i ens1f0np0
mlnx_qos -i ens1f0np0 --pfc 0,0,0,1,0,0,0,0
mlnx_qos -i ens1f0np0 --trust dscp
sysctl -w net.ipv4.tcp_ecn=1
```

注意：① RoCE 流量不走 netdev，所以 ethtool -S 里看不到 RoCE 的包/丢包，要看 IB sysfs 的 counters；② PFC 必须在主机和交换机两端都开，且优先级一致；③ trust state 要在开 SR-IOV 之前设置，否则 VF 继承不到；④ VLAN 子接口会生成新的 GID，选 GID 时别选错。

对应文档：OFED / RoCE https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/rdma-over-converged-ethernet-roce.md

对应文档：OFED / QoS https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/quality-of-service-qos.md

3.10 SR-IOV：先开固件，再开系统，再切模式

典型现象：echo 到 sriov_numvfs 报错；VF 出不来；切 switchdev 失败；VF 能 ping 但不通外网。

```
# 1) 固件层：开 SR-IOV 并设定 VF 数
mlxconfig -d /dev/mst/mt4123_pciconf0 set SRIOV_EN=1 NUM_OF_VFS=8
# reboot 或 mlxfwreset

# 2) 系统层：创建 VF
cat /sys/class/net/ens1f0np0/device/sriov_totalvfs
echo 4 > /sys/class/net/ens1f0np0/device/sriov_numvfs

# 3) VF 策略
ip link set ens1f0np0 vf 0 mac 00:11:22:33:44:55 trust on
ip link set ens1f0np0 vf 0 spoofchk on

# 4) switchdev / representor（先 unbind VF）
devlink dev eswitch set pci/0000:3b:00.0 mode switchdev
devlink port show
```

注意：① BIOS 要开 above 4G decoding、VT-d/IOMMU；② 切 switchdev 前必须 unbind 所有 VF；③ 单口最大 127 VF，但实际数量受 BAR 空间和固件限制；④ VF LAG 场景 NUM_OF_VFS 不要超过 64；⑤ trust/spoofchk 等策略在驱动重启后会丢，需要固化。

对应文档：OFED / SR-IOVhttps://networking-docs.nvidia.com/mlnxofedswum/24101140lts/single-root-io-virtualization-sr-iov.md

3.11 DPDK / OVS：mlx5 是 bifurcated，不要绑 vfio-pci

典型现象：照着 Intel 网卡的教程把 mlx5 绑到 vfio-pci，结果 DPDK 起不来，或者内核网络也断了；OVS 开了 hw-offload，但没有 representor。

```
# 错误做法（不要对 mlx5 做）：
# dpdk-devbind.py -b vfio-pci 0000:3b:00.0

# 正确做法：保留 mlx5_core，直接 DPDK 用 PCI BDF
dpdk-testpmd -a 0000:3b:00.0 -- -i

# OVS 硬件卸
ovs-vsctl set Open_vSwitch . other_config:hw-offload=true
systemctl restart openvswitch
ovs-vsctl add-br br0
ovs-vsctl add-port br0 ens1f0np0
ovs-vsctl add-port br0 pf0vf0
ovs-appctl dpctl/dump-flows type=offloaded
```

判读：mlx5 的 bifurcated 驱动让内核与 DPDK 共享同一个 PCI 设备，所以不需要 vfio-pci；DPDK 只接管它需要的队列，内核仍能收控制面/慢路径流量。OVS 卸载则依赖 switchdev + representor，并且 `tc-policy=skip_sw` 才能保证“只走硬件”。

对应文档：DPDK / MLX5 PMDhttps://doc.dpdk.org/guides/nics/mlx5.html

3.12 调优与监控：四个计数器来源要对上

典型现象：看 ethtool -S 没有丢包，但业务侧有重传；看网卡 counters 正常，但交换机侧有 PFC 风暴；不知道应该信哪个。

计数器来源	看什么	典型命令/路径
netdev / ethtool	L2/L3 包、字节、drop、crc、link down	`ethtool -S ens1f0np0`
RDMA / IB sysfs	RoCE 的 port/prio 计数、QP 错误	`/sys/class/infiniband/mlx5_0/ports/1/counters/`
物理层 / mlxlink	BER、FEC、眼图、误码	`mlxlink -d <mst> -c`
驱动/固件健康	health reporter、fatal、recovery	`devlink health show`、`dmesg`

```
tuned-adm profile network-latency
cat /proc/interrupts | grep -E "mlx5|ens1f0np0"
cat /sys/class/infiniband/mlx5_0/ports/1/counters/port_rcv_errors
cat /sys/class/infiniband/mlx5_0/ports/1/counters/port_xmit_discards
devlink health show
```

调优的顺序建议：BIOS（above 4G、VT-d、ASPM、电源策略）→ 内核 cmdline（iommu=pt、pcie_aspm=off）→ tuned profile → IRQ 亲和 → 队列/ring/coalesce → 协议栈 sysctl → 应用（NUMA、hugepages、memlock）。不要一上来就改 sysctl；很多“调优无效”其实是 BIOS 或 IRQ 亲和没做。

对应文档：性能调优 KBhttps://enterprise-support.nvidia.com/s/article/performance-tuning-for-mellanox-adapters

对应文档：mlx5 计数器 KBhttps://enterprise-support.nvidia.com/s/article/understanding-mlx5-linux-counters-and-status-parameters

> **本章小结**
>
> 12 个落点可以压缩成一句话：先认卡，再认栈；固件归 mlxconfig，驱动归 OFED，接口归 ethtool/sysfs，数据面归 verbs/DPDK/OVS，物理层归 mlxlink。每一类都有官方文档，不要靠猜。

第 4 章 所谓“文件系统参数”，在 CX6 上到底指什么

很多学员问：“CX6 的文件系统参数在哪里配？”严格说，网卡没有“文件系统”。大家真正想找的，是驱动和协议栈暴露出来的那组 sysfs 节点、/etc 下的配置文件、/dev 下的设备节点。它们分散在四个地方：/sys（运行时状态与控制）、/etc（持久化配置）、/dev（设备节点）、/proc（内核视图）。这一章把它们一次讲清。

> **先澄清一个歧义**
>
> 如果你说的“文件系统”是指 NVMe-oF、NFS-RDMA 这类存储协议，那属于存储层，不是网卡配置；入口在 OFED 手册的 Storage Protocols 章节和 SPDK/NVMe-oF 文档。本章讲的是“网卡与驱动相关的系统参数”。

4.1 配置落点总表

配置对象	CX6 的位置	说明	重启后
PCI 设备/驱动绑定	/sys/bus/pci/devices/<bdf>/	看 driver、enable、numa_node、msi_irqs	保持
netdev 存在与属性	/sys/class/net/<if>/	mtu、address、operstate、queues、statistics	保持
队列/ring/coalesce	/sys/class/net/<if>/queues/	运行时由 ethtool 改，sysfs 可读	丢失
SR-IOV 使能与 VF 数	/sys/class/net/<if>/device/sriov_numvfs	运行时创建 VF；上限看 sriov_totalvfs	丢失
VF 策略	/sys/class/net/<if>/device/sriov/<n>/	mac、vlan、trust、spoofcheck、max_tx_rate	丢失
RoCE GID 表	/sys/class/infiniband/mlx5_0/ports/1/gids/	每个 IP/VLAN 生成 GID；类型在 gid_attrs/types	随 IP 重建
RoCE 计数	/sys/class/infiniband/mlx5_0/ports/1/counters/	RoCE 流量不进入 netdev 计数	累计
RoCE 开关	/sys/devices/<bdf>/roce_enable	0/1 关闭/开启 RoCE	驱动重载后恢复
流量类/ToS	/sys/class/infiniband/mlx5_0/tc/1/traffic_class	Force DSCP/TCLASS	丢失
MFT 设备节点	/dev/mst/	mst start 创建；mlxconfig/mlxlink 使用	需 mst start
uverbs 设备	/dev/infiniband/	uverbs、rdma_cm、umad	由内核创建
OFED 服务/模块	/etc/infiniband/openib.conf	控制 OFED 启动时加载哪些模块/服务	保持

配置对象	CX6 的位置	说明	重启后
驱动模块参数	/etc/modprobe.d/*.conf	mlx5_core 参数、软依赖	保持（需 initramfs）
持久化命名	/etc/udev/rules.d/85-net-persistent-names.rules	按 MAC 固定接口名	保持
memlock 限制	/etc/security/limits.conf + limits.d/	RDMA 锁页内存；常设为 unlimited	保持（PAM）
网络协议栈	/etc/sysctl.d/*.conf	rmem/wmem、backlog、ecn、rp_filter 等	保持
L3 配置	/etc/NetworkManager/ 或 /etc/sysconfig/network-scripts/	IP、路由、VLAN、bond	保持
hugepages	/sys/kernel/mm/hugepages/、/dev/hugepages	DPDK/VM 大页	丢失（需 cmdline/sysctl）
调试节点	/sys/kernel/debug/mlx5/<bdf>/	steering、EQ、health 等；生产慎用	保持

4.2 八个必须会看的路径

（1）/sys/class/net/<if>/：接口的一切

```
ls /sys/class/net/ens1f0np0/
cat /sys/class/net/ens1f0np0/mtu
cat /sys/class/net/ens1f0np0/operstate
cat /sys/class/net/ens1f0np0/address
cat /sys/class/net/ens1f0np0/speed
ls /sys/class/net/ens1f0np0/queues/
ls -l /sys/class/net/ens1f0np0/device   # 指向 PCI 设备
```

用途：确认接口是否存在、MTU/状态/MAC、队列数、PCI 归属。device 是 symlink，顺着它就能找到 BDF，这是建立“netdev ↔ PCI”映射最快的方法。

（2）/sys/class/infiniband/mlx5_0/：RoCE 的主战场

```
ls /sys/class/infiniband/mlx5_0/ports/1/
cat /sys/class/infiniband/mlx5_0/ports/1/state
cat /sys/class/infiniband/mlx5_0/ports/1/link_layer
cat /sys/class/infiniband/mlx5_0/ports/1/rate
cat /sys/class/infiniband/mlx5_0/ports/1/gid_attrs/types/*
cat /sys/class/infiniband/mlx5_0/ports/1/counters/port_rcv_errors
cat /sys/class/infiniband/mlx5_0/ports/1/hw_counters/
```

用途：确认 RDMA 设备状态、链路层（Ethernet/InfiniBand）、速率、GID 类型、RoCE 计数。RoCE 的丢包/重传优先看 hw_counters 和 counters，不要只看 ethtool -S。

(3) /sys/class/net/<if>/device/sriov/：VF 的运行时配置

```
cat /sys/class/net/ens1f0np0/device/sriov_totalvfs
cat /sys/class/net/ens1f0np0/device/sriov_numvfs
ls /sys/class/net/ens1f0np0/device/sriov/0/
cat /sys/class/net/ens1f0np0/device/sriov/0/mac
cat /sys/class/net/ens1f0np0/device/sriov/0/trust
cat /sys/class/net/ens1f0np0/device/sriov/0/stats
```

用途：创建/查看 VF，设置 MAC/VLAN/trust/spoofchk/速率。注意这些是运行时配置，驱动重启后会回到默认值；要持久化，需要 udev 规则或 systemd 服务。

(4) /dev/mst/ 与 /dev/infiniband/：两类设备节点

```
ls -l /dev/mst/
mst status -v
ls -l /dev/infiniband/
# uverbs0、rdma_cm、umad0 等
```

用途：/dev/mst 是 MFT 的配置通道（mlxconfig、mlxlink、mlxfwmanager 用）；/dev/infiniband 是 verbs 的字符设备。前者需要 mst start，后者由内核自动创建。排障时“找不到设备”先分清楚是哪一类。

(5) /etc/infiniband/openib.conf：OFED 的总开关

```
cat /etc/infiniband/openib.conf
# 关注：OPENIBD_LOAD_MLX5_IB、OPENIBD_LOAD_IPOIB、OPENIBD_LOAD_SRP 等
systemctl status openibd
```

用途：控制 OFED 启动时加载哪些模块/服务。改完需要重启 openibd 或重启系统。inbox 驱动没有这个文件，这也是判断“是否装了 OFED”的一个侧证。

(6) /etc/modprobe.d/：驱动参数与软依赖

```
ls /etc/modprobe.d/
grep -R "mlx5" /etc/modprobe.d/ 2>/dev/null
# 示例：options mlx5_core num_of_groups=20
# 改完：dracut -f（或 update-initramfs -u），再重启
```

用途：给 mlx5_core 传模块参数（如 num_of_groups、probe_vf），或配置 bonding/8021q 的软依赖。注意：模块参数不是“万能调优”，很多参数改了会影响所有 PF/VF，生产环境先小范围验证。

(7) /etc/security/limits.conf：RDMA 的锁页内存

```
grep -R "memlock" /etc/security/limits.conf /etc/security/limits.d/ 2>/dev/null
# 建议：
# *  soft  memlock  unlimited
# *  hard  memlock  unlimited
ulimit -l
```

用途：RDMA 应用注册内存需要锁页；memlock 太小会报 “Cannot allocate memory” 或注册 MR 失败。容器场景还要在容器内单独设置，或使用 privileged/ulimits。

（8）/etc/sysctl.d/：网络协议栈参数

```
ls /etc/sysctl.d/
sysctl -a | grep -E "rmem|wmem|backlog|tcp_ecn|rp_filter"
# 示例：
# net.core.rmem_max = 134217728
# net.core.netdev_max_backlog = 250000
# net.ipv4.tcp_ecn = 1
```

用途：TCP/UDP 缓冲区、backlog、ECN、ARP 行为、rp_filter 等。注意：RoCE 流量不走内核协议栈，所以这些参数主要影响 TCP/IP 与控制面，不要指望它们能“调好 RoCE”。

4.3 持久化矩阵：哪些重启会丢

配置	设置方式	重启后	固化方式
IP / 路由 / MTU	ip / nmcli	丢失	NetworkManager、systemd-networkd、ifcfg
队列 / ring / coalesce	ethtool -L/-G/-C	丢失	udev rule、NM dispatcher、systemd oneshot
SR-IOV VF 数	echo > sriov_numvfs	丢失	udev rule（ACTION==add）、systemd oneshot
VF MAC/VLAN/trust	ip link set ... vf	丢失	udev rule、systemd oneshot
SRIOV_EN / NUM_OF_VFS	mlxconfig set	保持	固件 NV 配置
RoCE 模式	cma_roce_mode	丢失	systemd oneshot
PFC / trust / ratelimit	mlnx_qos	丢失	systemd oneshot
memlock	limits.conf	保持	PAM（重新登录生效）
sysctl	sysctl.d	保持	systemd-sysctl
hugepages	sysfs / cmdline	丢失/保持	kernel cmdline 或 sysctl
接口命名	udev rules	保持	udev
模块参数	modprobe.d	保持	重建 initramfs

4.4 一个最小固化示例：RoCE + PFC

下面这个 systemd 服务把“RoCEv2 + PFC”在每次开机后重新应用。它不依赖厂商工具，全部用标准命令，适合作为课程演示的“把运行时配置变成持久配置”的模板。

```
# /etc/systemd/system/cx6-roce.service
[Unit]
Description=Apply CX6 RoCE/PFC settings
After=network-online.target openibd.service
Wants=network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/sbin/cma_roce_mode -d mlx5_0 -p 1 -m 2
ExecStart=/usr/sbin/mlnx_qos -i ens1f0np0 --trust dscp
ExecStart=/usr/sbin/mlnx_qos -i ens1f0np0 --pfc 0,0,0,1,0,0,0,0

[Install]
WantedBy=multi-user.target
```

```
systemctl daemon-reload
systemctl enable --now cx6-roce.service
systemctl status cx6-roce.service
```

注意：PFC 的优先级必须和交换机一致；trust dscp 还是 pcp 取决于网络设计；如果开了 SR-IOV，trust state 要在 VF 创建之前设置。这个例子只做演示，生产环境请加错误处理和日志。

4.5 如果只记十个路径

#	路径	一句话用途
1	/sys/class/net/<if>/device	从 netdev 找到 PCI BDF
2	/sys/class/net/<if>/device/sriov_numvfs	创建/查看 VF
3	/sys/class/infiniband/mlx5_0/ports/1/	RoCE 状态、GID、计数
4	/dev/mst/	mlxconfig / mlxlink 的设备入口
5	/etc/infiniband/openib.conf	OFED 模块/服务总开关
6	/etc/modprobe.d/	驱动参数
7	/etc/security/limits.d/	memlock
8	/etc/sysctl.d/	TCP/IP 协议栈
9	/etc/udev/rules.d/	接口命名与持久化
10	/sys/kernel/debug/mlx5/<bdf>/	深度排障（慎用）

第 5 章 CX6 文档地图：一份可以直接照做的清单

这一章回答“到底该看哪些文档”。我们按“先总后分”组织：先给一张六层地图，再给 14 份核心文档，然后按任务给出索引，最后讲版本怎么选、旧文档怎么用。所有链接都在 2026-10 核对过；如果打不开，按第 5.5 节的检索技巧找同名新版。



图 4 CX6 文档的六层结构：从硬件手册到应用生态，每层回答不同的问题

5.1 核心 14 份文档

1. ConnectX-6 VPI 用户手册（硬件）

https://networking-docs.nvidia.com/connectx6vpihw/

解决：板卡接口、LED、供电、上架、规格、驱动安装索引、固件升级、排障。

重点：Interfaces / Hardware Installation / Driver Installation / Updating Adapter Firmware / Troubleshooting。

2. ConnectX-6 Dx 用户手册（硬件）

https://networking-docs.nvidia.com/connectx6dxhw/

解决：Dx 的 OPN、接口、规格、驱动安装、固件升级、监控、MAC 查找。

重点：Introduction（OPN 表）/ Interfaces / Specifications / Monitoring。

3. MLNX_OFED 24.10 LTS 用户手册（驱动栈，当前主线）

https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/

解决：安装、卸载、Secure Boot、性能调优入口，以及 Ethernet/RoCE/QoS/SR-IOV/OVS 的全部配置。

重点：Installation / Ethernet Network / Virtualization / Troubleshooting。

4. MLNX_OFED 5.9 用户手册（旧版，但语义最完整）

https://networking-docs.nvidia.com/mlnxofedswum/590560125/

解决：很多 24.10 手册里被精简掉的细节（GID 示例、RoCE LAG、VF 策略、QoS 映射）。

重点：RDMA over Converged Ethernet / Quality of Service / Single Root IO Virtualization。

5. MLNX_OFED → DOCA-OFED 迁移指南

https://networking-docs.nvidia.com/doca/archive/3-2-3/mlnx_ofed-to-doca-ofed-transition-guide

解决：为什么 OFED 改名、DOCA-Host 有哪些 profile、迁移步骤、时间线。

重点：What is DOCA-OFED / Transition Timeline / Installation Example。

6. DOCA 3.2.3 General Support（兼容矩阵）

https://networking-docs.nvidia.com/doca/archive/3-2-3/general-support

解决：哪些 OS/kernel 受支持、各卡捆绑的固件版本、DOCA 组件版本。

重点：Supported Platforms and NIC Firmware Versions / Supported Host OS。

7. NVIDIA Firmware Tools（MFT）用户手册

https://networking-docs.nvidia.com/mftswum/434118lts/

解决：mst、mlxfwmanager、mlxconfig、flint、mlxfwreset、mlxlink、mlxcables 的完整用法。

重点：mst Service / mlxfwmanager / mlxconfig / mlxlink / Cable Utilities。

8. mlxconfig：改固件配置

https://networking-docs.nvidia.com/mftswum/434118lts/using-mlxconfig

解决：查询/设置固件参数、SR-IOV、端口类型、备份/恢复、raw TLV。

重点：Using mlxconfig / mlxconfig Commands / MFT Supported Configurations and Parameters。

9. mlxlink：链路诊断

https://networking-docs.nvidia.com/mftswum/434118lts/mlxlink-utility

解决：链路状态、FEC、BER、眼图、模块信息、PRBS、PCIe 链路。

重点：Queries / Commands / Tool Usage with NIC vs. Switch。

10. mlxcables：光模块/线缆

https://networking-docs.nvidia.com/mftswum/434118lts/mlxcables-cables-tool

解决：光模块 EEPROM、DDM、寄存器读取、线缆固件升级。

重点：Query / DDM / read_reg / dump。

11. DPDK MLX5 PMD 文档

https://doc.dpdk.org/guides/nics/mlx5.html

解决：DPDK 下 mlx5 的编译、配置、Rx/Tx 调优、flow steering、representor、多进程。

重点：Configuration / Rx/Tx Tuning / Features & Support / Usage。

12. ConnectX-6 固件发布说明

https://docs.nvidia.com/networking/display/connectx6firmwarev20434100lts/changes+and+new+features

解决：20.43 分支的新功能、已知问题、修复、兼容的驱动/工具版本。

重点：Changes and New Features / Firmware Compatible Products / Known Issues。

13. ConnectX-6 Dx 固件发布说明

https://docs.nvidia.com/networking/display/connectx6dxfirmwarev22438002lts/changes+and+new+feature+history

解决：22.43 分支的新功能、已知问题、兼容矩阵。

重点：Changes and New Feature History / Firmware Compatible Products。

14. 性能调优与计数器 KB

https://enterprise-support.nvidia.com/s/article/performance-tuning-for-mellanox-adapters

解决：BIOS/内核/IRQ/队列/协议栈的调优顺序，以及 mlx5 计数器含义。

重点：Performance Tuning / Understanding mlx5 Linux Counters / ethtool Counters。

5.2 按任务找文档

你要做的事	先看这份	再看这份	关键章节
第一次上架、接线、确认规格	CX6 / Dx 用户手册	卡的特性表	Interfaces / Specifications
装驱动、选 OFED 版本	OFED 24.10 安装	DOCA General Support	Installation / Supported OS
升级固件、改固件参数	MFT 手册	固件发布说明	mlxfwmanager / mlxconfig
RoCE 不通、丢包、重传	OFED RoCE 章节	RoCE 部署 KB	GID / PFC / ECN / counters
PFC、ETS、DSCP 信任	OFED QoS 章节	mlnx_qos 用法	Trust / PFC / prio_tc
SR-IOV、VF、switchdev	OFED Virtualization	SR-IOV KVM KB	SR-IOV / VF 策略 / representor
OVS 硬件卸载	OFED OVS ASAP²	OVS 官方文档	SwitchDev / tc-policy / representor
DPDK 收发包	DPDK mlx5	OFED 性能调优	Configuration / Rx/Tx Tuning
NVMe-oF / NFS-RDMA	OFED Storage Protocols	SPDK / nvme-cli 文档	NVMe-oF Target Offload
性能上不去	性能调优 KB	OFED Performance Tuning	BIOS / IRQ / 队列 / sysctl
排障、看计数器	OFED Troubleshooting	计数器 KB	General / Ethernet / Performance

5.3 版本怎么选（2026 年的现实建议）

场景	推荐	理由	注意
新部署，OS 在 DOCA 3.x 支持列表	DOCA-OFED 3.2.x（doca-ofed profile）	持续新功能；与 MLNX_OFED 1:1 替代	3.2+ 用 DKMS 源码编译，NVIDIA 不再签驱动
存量 CX6 VPI，追求稳定	MLNX_OFED 24.10 LTS	最后一个独立版，维护到 2027-10	不再加新功能；2027-10 后需迁移
只要以太网，包越小越好	inbox mlx5_core 或 DOCA-RoCE	跟随内核，少装包	功能受内核版本限制；RoCE/卸载可能不全
需要 IPsec/TLS/MACsec	DOCA-OFED + CX6 Dx	加密卸载是 Dx 的核心能力	注意型号是否带 Crypto；带宽有上限

场景	推荐	理由	注意
DPDK / OVS 卸载	DOCA-OFED 或 inbox + DPDK	mlx5 PMD 在 DPDK 内，不依赖 OFED 全量	不要绑 vfio-pci；switchdev 要 representor

固件分支对照（截至 2026-10，具体以 DOCA General Support 与固件发布说明为准）：

卡	固件分支	DOCA 3.2.3 捆绑版本	主要文档
ConnectX-6 VPI	20.43.x	20.43.8016	CX6 用户手册 + CX6 固件 RN
ConnectX-6 Dx	22.4x	22.47.3576	CX6 Dx 用户手册 + Dx 固件 RN
ConnectX-6 Lx	26.4x	26.47.3576	CX6 Lx 用户手册 + Lx 固件 RN

5.4 旧文档还有用吗?

有用，而且 5.9 的 OFED 手册在很多细节上比 24.10 更完整。原因是 24.10 为了适配 DOCA 结构，把一部分“原理 + 示例”压缩了；而 5.9 仍保留了完整的 GID 表示例、RoCE LAG、VF 策略、QoS 映射流程。建议的用法是：

- **装驱动、查 OS 支持：**看 24.10 或 DOCA 3.x，不要看 5.9。
- **查 RoCE/QoS/SR-IOV 的“语义和示例”：**看 5.9 的对应章节，命令仍适用；把 ifconfig/vconfig 换成 ip。
- **查新功能、新参数：**看 24.10 + 固件 RN；5.9 里没有。
- **查工具用法：**永远看最新 MFT 手册，因为参数会增删。

一个常见的坑：5.9 手册里的示例用 ifconfig、vconfig、ethX；在新系统上这些命令可能没装，或者命名已经不是 ethX。把示例“翻译”成 ip/NetworkManager 的写法，再执行。

5.5 检索技巧：怎么在 30 秒内找到

技巧	示例	说明
精确标题	"ConnectX-6 Dx Ethernet Adapter Cards User Manual"	NVIDIA 文档标题很固定，精确搜命中率最高
限定站点	site:docs.nvidia.com ConnectX-6 RoCE	优先官方；中文博客只作线索
限定新平台	site:networking-docs.nvidia.com mlxconfig	新平台支持 .md，加载快
搜 KB	site:enterprise-support.nvidia.com mlx5 counters	配置示例/排障步骤多在 KB
加 PDF	"MLNX_OFED" "24.10" filetype:pdf	离线阅读/发给同事
用版本号	"MLNX_OFED 24.10" "RoCE"	避免搜到 4.7/5.0 的旧文

第 6 章　30 分钟上机检查单

这一章给一套“第一次拿到 CX6 机器”的顺序。目标不是把性能调到极致，而是在 30 分钟内回答五个问题：卡是哪张、驱动是哪套、链路通不通、RoCE 能不能跑、VF 能不能开。每一步都给出命令和判读标准。

#	步骤	命令	看什么 / 判读
1	认卡	`lspci -nn | grep -i mellanox`	15b3:101b/101d/101f；确认 VPI/Dx/Lx
2	认固件	`mst start && mlxfwmanager --query`	Device Type、PSID、FW 分支是否匹配
3	认驱动	`ethtool -i ens1f0np0`、`ofed_info -s`	driver=mlx5_core；OFED 版本是否存在
4	认接口	`ip -br link`、`ibdev2netdev`	PF netdev、mlx5_x 的映射
5	看链路	`ethtool ens1f0np0`、`mlxlink -d <mst> -m -c`	speed/width/FEC/BER；模块是否识别
6	配 IP	`ip addr add 192.0.2.1/24 dev ens1f0np0`	ping 对端；确认 MTU
7	看计数	`ethtool -S ens1f0np0 | head -40`	rx/tx、drop、crc、link_down 是否增长
8	验 RDMA	`ibv_devices`、`ibv_devinfo`、`rdma link show`	PORT_ACTIVE；link_layer=Ethernet
9	验 RoCE	`cma_roce_mode -d mlx5_0 -p 1 -m 2`、`ibv_rc_pingpong`	能建连；GID 类型为 v2
10	看 QoS	`mlnx_qos -i ens1f0np0`	PFC/trust/prio_tc 是否符合设计
11	开 SR-IOV	`mlxconfig q`、`sriov_totalvfs`、`sriov_numvfs`	SRIOV_EN=1；VF 能创建
12	做基线	`iperf3` / `perftest` / `mlxlink -c`	记录吞吐/延迟/BER，作为后续对比

6.1 五分钟：把“卡、驱动、口”对上

```
# 一条命令拿到 PCI 信息
lspci -nn | grep -i mellanox

# 一条命令拿到固件/PSID/型号
mst start >/dev/null 2>&1
mlxfwmanager --query

# 一条命令拿到 netdev ↔ ibdev
ibdev2netdev
ip -br link | grep -E "ens|enp"
```

如果这三条命令的信息对不上（例如 mlxfwmanager 显示 ConnectX6Dx，但 lspci 是 101b），先停下来查清楚，不要继续往下配。型号错，后面全错。

6.2 十分钟：看链路与物理层

```
ethtool ens1f0np0
ethtool -m ens1f0np0 | head -40
mlxlink -d /dev/mst/mt4123_pciconf0 -m -c
mlxcables -d /dev/mst/mt4123_pciconf0_cable_0 -q
```

判读顺序：① 有没有 link；② speed/width 是否是预期；③ FEC 是否匹配（25G/50G/100G/200G 要求不同）；④ BER/误码是否在阈值内；⑤ 光模块温度/功率是否正常。链路问题不要先怀疑驱动。

6.3 十五分钟：跑通 RoCE 与 SR-IOV

```
# RoCE
cma_roce_mode -d mlx5_0 -p 1 -m 2
ibv_rc_pingpong -d mlx5_0 -g 3        # GID 索引按实际表选

# SR-IOV（确认 BIOS 已开 above 4G + IOMMU）
cat /sys/class/net/ens1f0np0/device/sriov_totalvfs
echo 2 > /sys/class/net/ens1f0np0/device/sriov_numvfs
lspci | grep -i "virtual function"
ip link show ens1f0np0 | grep vf
```

RoCE 跑通的最低标准：ibv_rc_pingpong 能完成、没有立即报错、对端能看到 GID；SR-IOV 跑通的最低标准：VF 出现在 lspci、PF 的 ip link 能看到 vf 行、VF 能绑定驱动。更高阶的 PFC/ECN/性能，放到第二阶段。

6.4 异常速查

现象	先看	常见原因
lspci 看不到卡	BIOS、插槽、供电、dmesg	PCIe 未识别/供电不足/插槽故障
有卡但无 netdev	dmesg、`ibdev2netdev`、驱动绑定	mlx5_core 未加载/绑定错驱动
link down	`ethtool`、`mlxlink -m`、光模块	模块不匹配/光纤/交换机端口
ping 不通	IP/路由/ARP、`rp_filter`	IP 配错/网关/防火墙/反路由
RoCE 建连失败	`ibv_devinfo`、GID、`cma_roce_mode`	GID 选错/RoCE 模式/交换机 PFC
SR-IOV 创建失败	`sriov_totalvfs`、BIOS、dmesg	above 4G 未开/IOMMU 未开/BAR 不足
VF 不通	VF MAC/VLAN/trust、switchdev	VLAN 不匹配/trust 未开/representor 未配
性能低	队列/IRQ/NUMA/BIOS	单队列/跨 NUMA/ASPM/C-state

讲师提示

这一章最适合现场演示。建议提前准备一台机器，把“正确的输出”截图放在幻灯片里；现场只演示 3 条命令（认卡、mlxlink、ibv_devinfo），剩下的让学员照着检查单做。

第 7 章　十二个高频坑

这些坑来自真实环境，按“现象 → 根因 → 处理”列出。讲课时不必全讲，挑 5 个与学员场景最接近的展开。

#	现象	根因	处理
1	把 Dx 当 VPI，刷固件失败	型号/PSID 不匹配	先 `mlxfwmanager --query`，按 PSID 下固件
2	`ethtool -i` 版本与 `ofed_info` 对不上	inbox 与 OFED 混装	卸掉一套，只保留一套；重建 initramfs
3	只升驱动不升固件，功能不可用	新功能依赖新固件	按兼容矩阵同时升级驱动与固件
4	`mlxconfig` 改完不生效	没重启/没 `mlxfwreset`	`mlxfwreset query` 后 `reset`，或重启
5	SR-IOV 创建 VF 失败	BIOS 未开 above 4G/IOMMU	开 above 4G、VT-d、`intel_iommu=on`
6	切 switchdev 失败	VF 还被驱动占用	先 unbind 所有 VF，再切模式
7	RoCE 只配主机，交换机没配	PFC 不是端到端	交换机同优先级开 PFC；检查 DSCP/PCP 映射
8	在 `ethtool -S` 里找不到 RoCE 丢包	RoCE 不走 netdev	看 IB sysfs 的 counters/hw_counters
9	DPDK 把 mlx5 绑到 vfio-pci	照搬 Intel 网卡做法	保留 mlx5_core，用 `-a <bdf>`
10	`ethtool -L` 后 RSS 不均	改队列会重置 RSS 表	重新配置 `ethtool -X` 或 irq affinity
11	配了 QoS 但不生效	trust state 不对（PCP vs DSCP）	`mlnx_qos --trust dscp`；检查 VLAN/优先级
12	跨型号刷固件变砖	只看文件名，不看 PSID	永远按 PSID/Part Number 匹配；先备份

三条铁律：① 型号/PSID 不对，什么都不做；② 驱动与固件按兼容矩阵成对升级；③ 任何“改了不生效”，先问“这是运行时配置还是固件配置”。

第 8 章　90 分钟课程设计

这一章给讲师用。默认学员有 Linux 与网络基础，但不熟悉 NVIDIA 的文档体系；目标是“听完能自己找到文档、能按检查单上机”，而不是“记住所有命令”。

时间	模块	内容	形式	对应章节
0–5	开场	一个问题：你为什么找不到 CX6 的使用说明?	提问 + 投票	第 0 章
5–20	认知	CX6 家族、文档分层、平台迁移、版本绑定	讲授 + 图 1	第 1–2 章
20–40	差异地图	12 个落点，重点讲 1/3/5/9/10/11	讲授 + 图 2	第 3 章
40–55	现场演示	认卡、mlxlink、RoCE、SR-IOV、DPDK 五个 demo	实操 + 投屏	第 6 章
55–70	文档地图	14 份文档、按任务找文档、版本选择	讲授 + 图 4	第 5 章
70–82	排障演练	给 3 个故障现象，学员按检查单定位	分组练习	第 6–7 章
82–90	总结	三条铁律、课后作业、Q&A	讲授 + 答疑	第 7 章

8.1 五个现场演示

演示的目标不是“跑个分”，而是让学员看到“同一个物理口在不同层的名字和输出”。每个 demo 控制在 2–3 分钟。

```
# Demo 1：认卡（30 秒）
lspci -nn | grep -i mellanox
mlxfwmanager --query
ethtool -i ens1f0np0

# Demo 2：路与光模（60 秒）
ethtool ens1f0np0
mlxlink -d /dev/mst/mt4123_pciconf0 -m -c
mlxcables -d /dev/mst/mt4123_pciconf0_cable_0 --DDM

# Demo 3：RoCE（60 秒）
ibdev2netdev
cma_roce_mode -d mlx5_0 -p 1 -m 2
ibv_devinfo -v | head -40

# Demo 4：SR-IOV（60 秒）
cat /sys/class/net/ens1f0np0/device/sriov_totalvfs
echo 2 > /sys/class/net/ens1f0np0/device/sriov_numvfs
lspci | grep -i "virtual function"

# Demo 5：DPDK（60 秒）
dpdk-testpmd -a 0000:3b:00.0 -- -i
```

演示前把“正确的输出”截图放进备份幻灯片；现场如果卡住，直接切截图，不要在现场排障超过 2 分钟。另外，SR-IOV/DPDK 演示会改变机器状态，务必在实验机上进行。

8.2 五个互动问题

- “如果 lspci 看到 15b3:101d，你会打开哪本手册？为什么？”（答案：Dx 手册，不是 VPI。）
- “mlxconfig 改了 SRIOV_EN，为什么 ethtool 看不到变化？”（答案：固件配置，需重启/mlxfwreset；ethtool 看运行时。）
- “RoCE 丢包，为什么 ethtool -S 没有记录？”（答案：RoCE 不走 netdev，看 IB sysfs。）
- “VF 创建失败，你先查 BIOS 还是先查驱动？”（答案：先看 sriov_totalvfs 与 dmesg，再查 BIOS/IOMMU。）
- “DPDK 能不能把 mlx5 绑到 vfio-pci？”（答案：不要；mlx5 是 bifurcated。）

8.3 课后作业

- 在自己的机器上完成第 6 章的 12 步检查单，记录每一步的输出（截图或文本）。
- 画一张“你的机器”的映射图：PCI BDF → netdev → ibdev → VF/representor → MST。
- 从第 5 章挑 3 份文档，各读一个章节，写 5 行摘要：这份文档解决什么问题、关键命令、坑。

8.4 扩展为半天工作坊

加出来的时间	内容	产出
+30 分钟	RoCE/PFC 端到端实验（主机 + 交换机）	一张 PFC/ECN 配置对照表
+45 分钟	SR-IOV + switchdev + OVS 卸载	一个可复现的 OVS 硬件卸载 demo
+45 分钟	DPDK testpmd + 性能基线	一份吞吐/延迟/CPU 占用基线
+30 分钟	故障注入（错配 FEC、错配 VLAN、错配 PSID）	一份排障决策树

第 9 章 附录

附录 A 型号 / 固件 / 文档对照

项目	ConnectX-6 VPI	ConnectX-6 Dx	ConnectX-6 Lx
定位	IB + Ethernet 双栈	以太网 SmartNIC + 加密卸载	低功耗以太网
典型速率	200Gb/s HDR / 200GbE	200/100/50/25GbE	50/25GbE
PCIe	3.0/4.0 x16	4.0 x16	4.0 x8/x16
固件分支	20.43.x	22.4x	26.4x
DOCA 3.2.3 捆绑	20.43.8016	22.47.3576	26.47.3576
硬件手册	connectx6vpihw	connectx6dxhw	connectx6lxhw（如有）
典型 OPN	MCX653106A-HDAT	MCX623106AN-CDAT	MCX631102AN-ADAT

注：DOCA 3.2.3 的捆绑版本来自其 General Support 页面；固件分支会随 LTS 更新，实际以 `mlxfwmanager --query` 和固件发布说明为准。

附录 B 命令速查

识别与版本

```
lspci -nn | grep -i mellanox
mst start
mst status -v
mlxfwmanager --query
ethtool -i <if>
ofed_info -s
ibdev2netdev
rdma link show
```

固件与配置

```
mlxfwmanager -u -d <mst> -i <fw.mfa>
mlxfwreset -d <mst> query
mlxfwreset -d <mst> reset -y
mlxconfig -d <mst> q
mlxconfig -d <mst> show_confs
mlxconfig -d <mst> set SRIOV_EN=1 NUM_OF_VFS=8
mlxconfig -d <mst> -f backup.conf backup
```

链路与光模块

```
ethtool <if>
ethtool -m <if>
ethtool --show-fec <if>
mlxlink -d <mst> -m -c
mlxlink -d <mst> --show_fec
mlxcables -d <mst>_cable_0 -q
mlxcables -d <mst>_cable_0 --DDM
```

队列 / 中断 / offload

```
ethtool -l <if>; ethtool -L <if> combined 8
ethtool -g <if>; ethtool -G <if> rx 4096 tx 4096
ethtool -c <if>; ethtool -C <if> adaptive-rx off rx-usecs 32
ethtool -k <if>; ethtool -K <if> tso on
ethtool --show-priv-flags <if>
tc qdisc add dev <if> ingress
tc filter show dev <if> ingress
```

RoCE / QoS / SR-IOV

```
cma_roce_mode -d mlx5_0 -p 1 -m 2
cat /sys/class/infiniband/mlx5_0/ports/1/gid_attrs/types/*
mlnx_qos -i <if>
mlnx_qos -i <if> --pfc 0,0,0,1,0,0,0,0
mlnx_qos -i <if> --trust dscp
echo 4 > /sys/class/net/<if>/device/sriov_numvfs
ip link set <if> vf 0 trust on
devlink dev eswitch set pci/<bdf> mode switchdev
devlink port show
```

附录 C 术语表

术语	含义	为什么重要
PF / VF	Physical Function / Virtual Function	SR-IOV 的两个层级；PF 管策略，VF 给 VM
BDF	Bus:Device.Function，如 0000:3b:00.0	PCI 设备的唯一标识；所有映射的起点
netdev	Linux 网络接口，如 ens1f0np0	IP/ethtool/tc 操作的对象
ibdev	RDMA 设备，如 mlx5_0	verbs/RoCE 操作的对象
representor	VF 在主机侧的代理接口，如 pf0vf0	switchdev/OVS 卸载的关键
MST	Mellanox Software Tools 设备节点，如 /dev/mst/mt4123_pciconf0	mlxconfig/mlxlink 的入口
PSID	Product Serial ID，固件与型号的匹配标识	刷错固件的根本原因之一
OFED	OpenFabrics Enterprise Distribution	NVIDIA 的驱动/库/工具栈
DOCA-OFED	MLNX_OFED 的 1:1 替代	2024-10 后的新主线
RoCEv2	RDMA over Converged Ethernet v2（UDP 4791）	最常用的 RDMA over Ethernet
GID	Global ID，RoCE 的地址表项	选错 GID 就连不上
PFC	Priority Flow Control	RoCE 无损的前提；必须端到端

术语	含义	为什么重要
ECN	Explicit Congestion Notification	拥塞控制；与 PFC 配合
DCBX	Data Center Bridging Exchange	QoS 参数的自动协商
ASAP²	Accelerated Switching And Packet Processing	OVS 硬件卸载
bifurcated	内核与 DPDK 共享同一 PCI 设备	mlx5 不需要绑 vfio-pci
CQE	Completion Queue Entry	收包完成事件；压缩可省 PCIe 带宽
LTS	Long Term Support	稳定分支；生产优先

附录 D 随讲义文件

- **硬件手册（离线）：** references/ NVIDIA_ConnectX-6_IB_Ethernet_Adapter_Cards_User_Manual.pdf
- **硬件手册（离线）：** references/ NVIDIA_ConnectX-6_Dx_Ethernet_Adapter_Cards_User_Manual.pdf
- **文档地图（可点击）：** CX6 文档地图.md
- **幻灯片：** CX6_vs_自研网卡_讲座.pptx

附录 E 参考资料

以下链接均在 2026-10 核对。NVIDIA 文档有版本选择器；如果链接失效，按标题搜索同名新版。

· NVIDIA Networking 文档总入口：https://docs.nvidia.com/networking/

· NVIDIA 新文档平台：https://networking-docs.nvidia.com/

· ConnectX-6 VPI 用户手册：https://networking-docs.nvidia.com/connectx6vpihw/

· ConnectX-6 Dx 用户手册：https://networking-docs.nvidia.com/connectx6dxhw/

· MLNX_OFED 24.10 LTS：https://networking-docs.nvidia.com/mlnxofedswum/24101140lts/

· MLNX_OFED 5.9（旧版，语义完整）：https://networking-docs.nvidia.com/mlnxofedswum/590560125/

· MLNX_OFED → DOCA-OFED 迁移指南：https://networking-docs.nvidia.com/doca/archive/3-2-3/mlnx_ofed-to-doca-ofed-transition-guide

· DOCA 3.2.3 General Support：https://networking-docs.nvidia.com/doca/archive/3-2-3/general-support

· MFT 4.34.1 用户手册：https://networking-docs.nvidia.com/mftswum/434118lts/

· mlxconfig：https://networking-docs.nvidia.com/mftswum/434118lts/using-mlxconfig

· mlxlink：https://networking-docs.nvidia.com/mftswum/434118lts/mlxlink-utility

· mlxcables：https://networking-docs.nvidia.com/mftswum/434118lts/mlxcables-cables-tool

· DPDK MLX5 PMD：https://doc.dpdk.org/guides/nics/mlx5.html

· 性能调优 KB：https://enterprise-support.nvidia.com/s/article/performance-tuning-for-mellanox-adapters

· RoCE 部署示例 KB：https://enterprise-support.nvidia.com/s/article/recommended-network-configuration-examples-for-roce-deployment

· mlx5 计数器 KB：https://enterprise-support.nvidia.com/s/article/understanding-mlx5-linux-counters-and-status-parameters